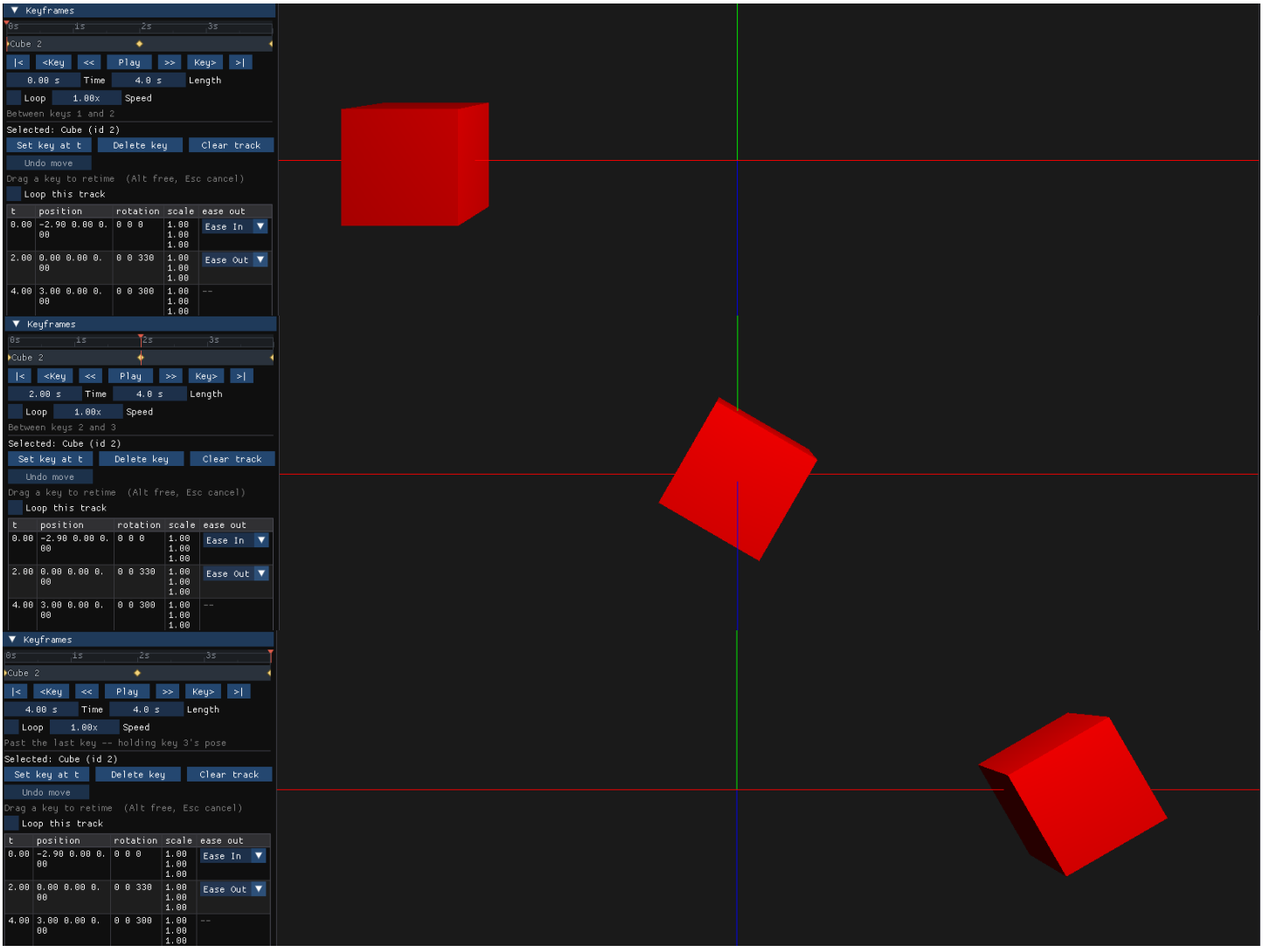
CALIFORNIA STATE UNIVERSITY
FULLERTON

Assignment 4

CPSC 484 – Principles of Computer Graphics
Assignment 4: Keyframe Animation
Fall 2026
Professor: Dave Garcia-Gomez
Version: 2026-10-15



1 Getting Started

In this assignment, you will extend the 3D Model Scene Editor developed in the previous assignments by implementing keyframe animation. You will record important poses at selected times and compute the poses between them so that objects can translate, rotate, and change scale during playback.

CALIFORNIA STATE UNIVERSITY FULLERTON

Assignment 4

A keyframe stores a shape's position, Euler rotation, and scale at one instant. A track stores the keys for one shape. The editor asks each track for its pose at the current time, then applies that pose to the shape. Your implementation must answer this question for any time, including when the user scrubs backward or jumps directly to another part of the timeline.

The primary objectives of this assignment are to:

- Maintain keyframes in chronological order without duplicate times;
- Insert, delete, and retime keys while preserving the identity of a dragged key;
- Interpolate position and scale and rotate each Euler angle along the shorter direction;
- Apply easing curves and support clamping and per-track looping; and
- Create and save an animated scene that demonstrates the completed functionality.

Before beginning, review **include/Keyframe.h**, **src/Keyframe.cpp**, and **src/AnimationTimeline.cpp**. The header describes the function contracts. The timeline, animation panel, playback controls, and scene persistence are already provided. Your responsibility is to complete the seven functions in Keyframe.cpp rather than redesigning the surrounding systems.

2 Building the Editor and Using Solution Libraries (Revisited)

As you work through the assignments, you will sometimes encounter functionality that was introduced in an earlier assignment but that you have not yet completed. Rather than requiring you to stop working until that functionality is finished, the editor supports **solution libraries**.

A solution library is a precompiled version of one part of the editor. You can add a supplied library to your project and tell the build system to use that completed implementation instead of your unfinished implementation.

This allows you to continue working on the current assignment while still using functionality from a previous assignment. This section is similar, but not exactly, the same as Assignment 2 and 3 and is included just for your reference.

2.1 Building the Editor

From the root directory of the editor project, extract the project if necessary and build it with the following commands:

```
tar xvzf editor.tar.gz
cd editor
make
```

If the build succeeds, an executable named editor will be created. Run the executable from the project directory:

```
./editor
```

You should run the editor from the project directory because the application loads resources such as shaders using paths relative to the current working directory.

2.2 Cleaning and Rebuilding

When you make changes to your source code, make will normally rebuild only the files that need to be recompiled.

If you want to remove the compiled object files and rebuild the editor from scratch, use:

```
make clean
```

Then rebuild the editor:

```
make
```

You can also combine the commands:

```
make clean && make
```

You should use make clean whenever you change which implementation of a topic is being used. This is especially important when switching between your implementation and a supplied solution library.

2.3 Using a Solution Library

Some functionality is implemented in later and earlier assignments. To allow you to use that functionality while working on your current assignment, solution libraries may be provided for your development environment.

Solution libraries are precompiled `.a` files. **Because compiled libraries are specific to the operating system, CPU architecture, and compiler/toolchain, you must use the library provided for your environment.** Do not copy a library from a different environment.

For example, if you want to use the completed camera implementation, you may be provided with:

```
libcamera.a
```

Step 1: Obtain the Library for Your Environment

Your instructor will provide the appropriate solution library separately based on your development environment. For example, the library provided for Linux may be different from the library provided for Windows or macOS.

Make sure you have the correct `.a` file for your environment before continuing.

Step 2: Copy the Library into `lib/solution/`

From the root directory of your editor project, copy the provided `.a` file into the `lib/solution/` directory.

Your directory should look similar to:

```
editor/
├──── lib/
│     └──── solution/
│           └──── libcamera.a
├──── src/
├──── Makefile
└──── ...
```

On Linux or macOS, you can copy the file using:

```
cp /path/to/libcamera.a lib/solution/
```

For example, if `libcamera.a` is in your Downloads directory:

```
cp ~/Downloads/libcamera.a lib/solution/
```

On Windows using MSYS2/MinGW, you can use:

```
cp /path/to/libcamera.a lib/solution/
```

or copy the file using Windows File Explorer and place it directly in:

```
editor\lib\solution\
```

Do not add the `.a` file manually to the `Makefile` or to `LIBS`. The build system will automatically locate and link the appropriate solution library when you use the corresponding build option.

Step 3: Build Using the Solution Library

Solution libraries are selected using a Make variable corresponding to the functionality they provide.

For example, to use the camera solution:

```
make clean
make CAMERA=solution
```

Then run the editor:

```
./editor
```

The CAMERA=solution option tells the build system to use the provided libcamera.a implementation instead of the camera implementation in your source tree.

For example, the following command:

```
make TRANSFORMS=solution
```

would use the provided transformation solution library instead.

The available solution libraries and their corresponding build options will be identified in the assignment instructions.

Important: Solution Libraries Are Environment-Specific

The source code for the assignment is shared across development environments, but the compiled `.a` libraries are not necessarily interchangeable.

For example:

```
Linux library       → Use with Linux
Windows library     → Use with Windows/MSYS2
macOS ARM library   → Use with Apple Silicon macOS
macOS Intel library → Use with Intel macOS
```

Always use the library supplied for your operating system, CPU architecture, and development environment. If you use an incompatible library, the linker may report errors or the resulting program may not run.

You do **not** need to modify the source code or Makefile to use a solution library. Copy the appropriate `.a` file into `lib/solution/` and use the corresponding `TOPIC=solution` build option.

Available Solution Libraries

The editor supports several optional solution libraries. These libraries allow you to temporarily use an instructor-provided implementation of a topic that you have not completed yet, so that you can continue working on later functionality.

For example, camera controls are part of a later assignment. While working on Assignment 2, you may use the provided camera solution library so that you can move around the scene while testing your transformations and geometry.

The editor currently recognizes the following solution libraries:

Functionality	Build Option	Library File
Model Transformations	TRANSFORMS=solution	libtransforms.a
Procedural Geometry and Normals	GEOMETRY=solution	libgeometry.a
OBJ Parsing	OBJPARSER=solution	libobjparser.a
Orbit, Pan, and Zoom Camera Controls	CAMERA=solution	libcamera.a
VR Camera / View Composition	VRCAMERA=solution	libvrcamera.a
Curves and Surfaces	CURVES=solution	libcurves.a

Functionality	Build Option	Library File
Skeletal Skinning	SKINNING=solution	libskinning.a
Keyframe Animation	ANIMATION=solution	libanimation.a
Texture Coordinates and Sampling	TEXTURES=solution	libtextures.a
Phong Lighting	LIGHTING=solution	liblighting.a
Ray-Tracer Shading	RAYTRACER=solution	libraytracer.a
Ray Intersections and BVH	BVH=solution	libbvh.a
Particle Integration and Forces	PARTICLES=solution	libparticles.a

Important: You should only use solution libraries that have been provided by your instructor. Some of the libraries listed above correspond to future assignments and may not be distributed until later in the semester.

2.4 Building the Solution Binary for Assignment 4

Assignment 4's solution file should include your code from Assignment 2 and 3 along with the solution files for Assignment 4. To build a solution binary from scratch, use the following libraries:

```
lib/solution/libtransforms.a
lib/solution/libgeometry.a
lib/solution/libobjparser.a
lib/solution/libcamera.a
lib/solution/libanimation.a
```

If you want to build a solution binary with your existing code, all you will need is the following libraries:

```
lib/solution/libanimation.a
```

Then clean your previous build:

```
make clean
```

Build the editor using the animation solution:

```
make ANIMATION=solution
```

To build the binary with Assignment 2 and 3 solutions as well:

```
make GEOMETRY=solution TRANSFORMS=solution CAMERA=solution
OBJPARSER=solution ANIMATION=solution
```

Run the editor:

```
./editor
```

To work on Assignment 4, just include the solutions from Assignments 2 and 3, or use your complete code from the completed assignments. The editor will now use the previous completed solution while the rest of the application continues to use your own source code, or your other solutions files specified.

This allows you to work on Assignment 4 while using all the functionalities of the previous assignments.

2.5 Returning to Your Own Implementation

Solution libraries are optional. If you want to return to your own implementation, remove the solution option from the build command.

For example, after using:

```
make ANIMATION=solution
```

return to your own camera implementation with:

```
make clean
make
```

The normal make command uses the source code in your project rather than the animation solution library.

2.6 Multiple Solution Libraries

You can use more than one solution library at the same time.

For example:

```
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

This tells the build system to use the supplied implementations for both transformations and geometry while continuing to compile all other topics from your source code.

Solution libraries can therefore be mixed with your own implementations. You do not have to replace the entire editor with a solution version.

For example:

```
make clean
make CAMERA=solution
```

means:

Camera	→	supplied solution library
Everything else	→	your source code

Similarly:

```
make clean
make TRANSFORMS=solution CAMERA=solution
```

means:

Transformations	→	supplied solution library
Camera	→	supplied solution library
Everything else	→	your source code

2.7 Important: Clean Before Switching Implementations

Always clean the project before changing a topic between your implementation and a solution library:

```
make clean
```

For example, if you first build with:

```
make CAMERA=solution
```

and later want to use your own implementation, do:

```
make clean
make
```

Likewise, if you are using your own implementation and want to switch to the solution:

```
make clean
make CAMERA=solution
```

Cleaning is important because previously compiled object files can remain in the project and interfere with the new build configuration.

2.8 Building a Full Solution Reference Editor

If you have been provided with all available solution libraries, you may build a fully functional version of the editor to use as a **reference while completing your assignments**.

This reference build can be useful when you want to compare your implementation against the completed behavior of the editor. For example, you can use it to check how a shape should render, how a control should respond, or how a later feature is expected to behave.

First, make sure all supplied `.a` files are located in:

```
lib/solution/
```

Then clean the project:

```
make clean
```

Build the editor using all available solution libraries:

```
make \
  TRANSFORMS=solution \
  GEOMETRY=solution \
  OBJPARSER=solution \
  CAMERA=solution \
  VRCAMERA=solution \
  CURVES=solution \
  SKINNING=solution \
  ANIMATION=solution \
  TEXTURES=solution \
  LIGHTING=solution \
  RAYTRACER=solution \
  BVH=solution \
  PARTICLES=solution
```

Then run:

```
./editor
```

This creates a version of the editor that uses the supplied solution implementation for every supported topic. The build system currently recognizes 13 solution topics.

You may use this full solution build as a **visual and behavioral reference** while developing your own code. For example, if you are unsure how your Cube, Sphere, transformations, camera controls, or another feature should behave, you can run the reference build and compare it against your own implementation.

The reference build does **not** replace the work required for the assignment. You are still responsible for implementing and submitting the source code required for your current assignment. Do not submit the solution libraries as your own implementation.

Also make sure that all solution libraries match your development environment. The `.a` files are specific to the operating system, CPU architecture, and compiler/toolchain being used.

2.9 Solution Libraries Do Not Replace Your Source Code

Using a solution library does **not** mean that you should copy someone else's source code into your project. The library is compiled code that is linked into your executable during the build process. Your source files remain in your project and can continue to be modified normally.

For example, when you build with:

```
make TRANSFORMS=solution GEOMETRY=solution
```

you can still edit and test your Assignment 4 files such as:

`Keyframe.cpp`
`AnimationTimeline.cpp`

The previous assignment's functionality is simply being supplied by the library while you work on the rest of the assignment.

2.10 Summary

The most common commands you will use are:

```
# Normal build
make

# Clean compiled files
make clean

# Clean and rebuild
make clean && make

# Build using the camera solution
make clean
make CAMERA=solution

# Build using the animation solution
make clean
make ANIMATION=solution

# Build using multiple solutions
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

Remember: Solution libraries allow you to use completed functionality without replacing your own source code. They are provided so that you can continue making progress when a previous or future assignment component is not yet implemented.

3. Assignment Overview

Everything you have built so far has been still. This assignment makes it move.

The idea is older than computer graphics: an animator draws the important poses, the **key** frames, and someone else fills in the frames between them. Here you are the someone else. The editor records a shape's transform at a moment in time; your code answers the only question playback ever asks:

> **What pose is this shape in at time t?**

That question has to be answerable for **any t**, instantly, in any order. This is the deep difference between keyframe animation and the particle systems: a particle system integrates forward and cannot be asked about an arbitrary time, which is why you cannot scrub one backwards. A keyframe track is a pure function of t. That is why the timeline in this editor can be dragged anywhere, including backwards, and why the whole thing works at all.

You will write two halves, and they are two halves of one idea:

Storage keeps the keys in order and unique in time. **Evaluation** relies on exactly that to interpolate between them.

They are in one file on purpose. A bug in the first half shows up in the second.

3.1 Functionality

The assignment has two related implementation tasks. **Storage** maintains a sorted collection of keys with unique times. **Evaluation** relies on that invariant to locate the interval containing a requested time and calculate its pose. A storage error can therefore appear as an interpolation error: an unsorted track selects the wrong interval, and duplicate times can create a zero denominator.

The starter compiles and the timeline controls are active, but evaluation returns false, so scrubbing does not animate shapes. Other stubs append keys without sorting, remove the first nearby key instead of the nearest, retain the old index when retiming, and treat every easing mode as linear. These are the behaviors you will replace.

3.2 Defined Scope

The files you will be working on, and the section in those files that you will be working on are as follows:

Half	Source file	Method to be completed	What does it do
Storage	`Keyframe.cpp`	`AnimationTrack::addKey()`	`insert, keeping the list sorted and unique in time`
Storage	`Keyframe.cpp`	`AnimationTrack::removeKeyNear()`	`erase the nearest key within a tolerance`
Storage	`Keyframe.cpp`	`AnimationTrack::moveKey()`	`retime a key and report where it ended up`
Evaluation	`Keyframe.cpp`	`applyEasing()`	`remap time through a curve`
Evaluation	`Keyframe.cpp`	`lerpVec3()`	`linear interpolation`
Evaluation	`Keyframe.cpp`	`lerpEulerDegrees()`	`rotation the short way round, per axis`
Evaluation	`Keyframe.cpp`	`AnimationTrack::evaluate()`	`the pose at t — the function everything else exists for`

Keep the existing public interfaces and data structures. A Keyframe contains time, position, rotation in degrees, scale, and easing. **AnimationTrack** contains **shapeId**, keys, and a loop flag. Read the existing declarations before changing any implementation.

Some of the important aspects of the code that is important to review and understand before you begin any work:

Source File	Method to be Completed
The timeline strip	The scrubbable bar at the top of the Animation panel. Tracks appear as you key objects, keys show as diamonds, and you can drag a diamond to retime it.
The transport	Play, frame step, jump to next/previous key, loop, speed, and a typeable time field.

keyShapeAt()	Captures a shape's current position, rotation and scale as a key. This is what **Set key at t** calls.
apply()	Asks every track for its pose at the current time and writes the results onto the shapes. It calls your **evaluate()**
Scene save/load	The **[track]** sections in a **.scene** file. Your animation is stored there, which matters, because the scene file is what you submit.

**The full contract for every one of them is written out in `include/Keyframe.h`. Read that header before you start. It is the specification; this handout is the explanation.

3.2.1 Where you are starting from

Launch the editor, insert a shape, move it, press **Set key at t**, move the playhead, move the shape, key it again. You will see a track appear with two diamonds on it, and you can drag them around.

And nothing will move when you scrub.

That is the starting point, and it is worth understanding **why**: the stubs are all working naive implementations, not empty holes. The timeline is completely alive. What is missing is the interpolation, and what is wrong is subtle:

- keys added out of order are not sorted
- keying the same instant twice makes two keys, not one
- dragging a key past its neighbour starts dragging a different key
- every easing curve behaves as linear

Every one of those is visible in the editor, and every one is yours.

3.3 Provided Components

AnimationTimeline.cpp supplies track lookup, **keyShapeAt()**, applying poses to shapes, playback, and timeline management. The animation panel supplies the scrubbable timeline, key diamonds, dragging, transport controls, time entry, and easing selection. Scene save/load preserves animation tracks. You do not need to implement a new timeline, animation controller, or file format.

Assignment 4

Keyframe animation is separate from the particle simulation code in AnimationController.cpp. A keyframe track evaluates a pose directly from time; it does not integrate a simulation forward frame by frame. Particle forces and numerical integration are outside this assignment and this course.

4 Part 1 - Keyframe Storage

Before the editor can calculate how a shape moves between poses, it needs a reliable way to store and manage those poses. In this part of the assignment, you will implement three functions that add, remove, and move keyframes within a shape's animation track. Each function must preserve the same rule:

The keys are sorted by time, and no two share an instant.

This rule is called an *invariant*: a condition that must remain true whenever an operation finishes. The evaluation code you will write in Part 2 relies on it to find the neighboring keys that surround a requested time. If the keys are out of order, evaluation may select the wrong pair. If two keys occupy the same instant, calculating the fraction between them can result in division by zero.

Because you are implementing both storage and evaluation, you may encounter an animation problem whose cause lies in an earlier storage operation. Keep this relationship in mind as you develop and test your code.

4.1 Maintaining the Track Invariant

After every insertion, removal, or move, the track must contain an ordered sequence of keys with a single pose at each keyed time. The user should be able to create poses in any order without having to arrange the underlying data manually.

Each key contains more than a time and position. It also stores rotation, scale, and an easing setting. When inserting, replacing, or moving a key, preserve all of these properties so that the pose and its timing behavior stay together.

Review the function declarations and default tolerances in Keyframe.h before beginning. Keep the existing interfaces so that your implementation continues to work with the timeline and animation panel.

4.2 Adding a Key

Implement AnimationTrack::addKey(const Keyframe& k) to insert a key into its correct chronological position. The user may create the final pose first and then return to an earlier time to add another pose, so you cannot assume that new keys belong at the end of the track.

If a key already exists at the supplied time, replace it with the new key. This allows the user to revise a pose by setting another key at the same instant. The replacement should include the complete key, including its easing setting, and should leave the number of keys unchanged.

Replacing an existing key also prevents ambiguity during evaluation. The interpolation parameter is calculated from two neighboring key times:

$$u = \frac{(t - \text{a.time})}{(\text{b.time} - \text{a.time})}$$

If those times are equal, the denominator is zero. Maintaining unique key times prevents this situation from being introduced through normal editing.

To test your implementation, add several keys out of order and verify that the stored sequence remains chronological. Then change a shape's pose and add another key at an existing time. The new pose should replace the old one without creating an additional key.

4.3 Removing the Nearest Key

Implement AnimationTrack::removeKeyNear(float t, float eps) to remove the key closest to the requested time, provided that it falls within the supplied tolerance. Return true when a key is removed. If the track is empty or no key is close enough, leave the track unchanged and return false.

The tolerance makes deletion practical in the editor because the playhead will not always land exactly on a key. The **Delete key** control passes a tolerance of 0.05 seconds, allowing it to find a nearby key even when the selected time is slightly offset.

More than one key may fall within that tolerance, so your function must compare their distances rather than remove the first match. For example, suppose the track contains keys at 1.00 and 1.04 seconds. A deletion request at 1.03 seconds with eps = 0.05 includes both keys, but the key at 1.04 seconds is closer and should be removed.

4.4 Moving a Key

Implement AnimationTrack::moveKey(size_t index, float t, float eps) to change the time of an existing key while keeping the track ordered. If index does not identify a valid key, return -1 without modifying the track.

Otherwise, clamp the destination time to zero or greater, move the selected key, and return its position in the reordered vector.

Returning the new index is essential to making key dragging work correctly. When the user drags a key diamond past a neighboring diamond, the keys change order in the vector. The caller in Renderer.cpp uses the index you return to continue tracking the same keyframe. If you return the original index, the next part of the drag may act on a different key, causing the selected pose to appear to change midway through the gesture. Before checking for a key at the destination time, lift the moving key out of the collection while preserving its pose and easing setting. Otherwise, the collision search may match the key against itself and remove the very key you intended to move.

If another key occupies the destination within the supplied tolerance, replace that key with the moved key. This follows the same rule as addKey: one instant should contain one pose, and the user's latest edit determines which pose is retained. When a move replaces another key, the track will contain one fewer key afterward.

Take care if you remove destination keys while iterating through the vector. Erasing an element shifts the following elements into earlier positions, so advancing the index immediately after an erase can skip an element that still needs to be checked. Once the move is complete, verify that the track remains sorted and that the returned index identifies the moved key.

Three traps, in the order you will meet them:

1. **Returning the new index is the entire point of this function** - Dragging a key past its neighbor re-sorts the vector underneath the drag. The caller in `Renderer.cpp` stores what you return so it keeps hold of the same keyframe. Return the index you were given and the drag silently jumps to a different key halfway through the gesture, which looks like the UI is broken, not your code.

2. **Lift the key out before looking for collisions** - If you sweep the list for a key at the destination time while the moved key is still in it, it matches itself, and you delete the thing you are moving.

3. **Dropping onto an existing key replaces it** - Same rule as **addKey** for the same reason. The moved key wins and the list gets shorter. If you erase while iterating, this will only advance your index when you haven't erased yet.

5 Part 2 - Interpolation and Evaluation

Once keyframes are stored correctly, the editor needs to determine how a shape should appear between them. In this part, you will implement interpolation for position, rotation, and scale, then combine these calculations with easing curves to evaluate a complete pose at any requested time. Your implementation will also handle track boundaries and looping so that playback, direct time entry, and scrubbing produce consistent results.

5.1 Linear Interpolation

Begin by implementing **lerpVec3(a, b, u)**, which calculates a value between two three-dimensional vectors. The parameter **u** describes how far the interpolation has progressed from **a** toward **b**: a value of 0 represents the starting vector, a value of 1 represents the ending vector, and a value of 0.5 represents the midpoint.

Use the following expression to interpolate each component of the vector:

$$value = a + u * (b - a)$$

For example, if a shape moves from (0, 0, 0) to (4, 2, 0), its position halfway through the interval will be (2, 1, 0). The same calculation works for scale, so you will use this helper for both properties when evaluating a pose.

Before performing the interpolation, clamp u to the range [0, 1]. If the supplied value is less than 0, return a; if it is greater than 1, return b. This ensures that the function holds the endpoint values instead of extending the motion beyond them.

5.2 Euler Rotation Interpolation

Rotation requires a little more care because angles repeat after a full turn. Implement lerpEulerDegrees(a, b, u) to interpolate the shape's rotation about each axis, with all angles expressed in degrees.

Consider a rotation from 350 degrees to 10 degrees. Interpolating those numbers directly would turn the shape backward through 340 degrees, even though the ending orientation is only 20 degrees ahead of the starting orientation. Your function should recognize this shorter route and rotate forward through the zero-degree boundary.

To achieve this, calculate the angular difference for each axis and wrap that difference into a half-turn range before interpolating. Your calculation should also work with negative angles and angles greater than 360 degrees. When the difference is exactly 180 degrees, the two directions have equal length.

As with linear interpolation, clamp **u** to **[0, 1]** and return the original endpoint values when u reaches 0 or 1. You do not need to normalize every intermediate result to **[0, 360]**. For example, an intermediate angle of 360 degrees represents the same orientation as 0 degrees and is a valid result.

5.3 Evaluating a Track

The interpolation helpers calculate individual properties, but the editor needs a complete pose for a shape at a particular time. Implement `AnimationTrack::evaluate(float t, Keyframe& out)` const to provide that pose. The function should write the result into out and return true whenever the track contains enough information to produce a pose.

Begin by handling tracks with no keys or only one key. An empty track has no pose to return, so the function should return false without changing out. A track with one key should hold that key's pose at every requested time, with **out.time** set to **t**.

For tracks with two or more keys, check whether per-track looping is enabled. If the requested time falls outside the keyed range and the track has a positive duration, wrap the time into that range before continuing. **Section 5.4** explains this behavior in more detail.

Once the time has been adjusted, handle the boundaries. A request at or before the first key should return the first pose, while a request at or after the last key should return the last pose. The track should hold these poses rather than extrapolate beyond them.

If the requested time falls between keys, find the consecutive keys **a** and **b** that surround it. Calculate the fraction of the interval that has elapsed using:

$$u = (t - a.time) / (b.time - a.time)$$

Pass this fraction through **applyEasing(a.easing, u)**, then use the eased value to interpolate position, rotation, and scale. Apply easing once so that all three properties follow the same timing.

Finally, set **out.time** to the time actually evaluated, including any adjustment for looping. For an interior segment, also copy the starting key's easing setting into **out.easing**. Although correctly stored keys should have distinct times, guard against a zero-duration interval so that an invalid pair cannot cause division by zero.

5.4 Looping and Boundary Behavior

A looping track repeats over the interval between its first and last keys. This interval does not necessarily begin at zero, so your implementation must use the actual key times when determining where a repeated cycle begins.

For example, suppose a track has its first key at 2 seconds and its last key at 5 seconds. Its duration is 3 seconds. A request at 6 seconds should wrap to 3 seconds, while a request at 1 second should wrap to 4 seconds. Looping must therefore work for requests on either side of the keyed range. If you use a remainder operation, remember that a negative input may produce a negative remainder that needs adjustment.

Only wrap a time that falls outside the range. A request at exactly 5 seconds in this example should return the final pose. Also avoid attempting to wrap a track with no positive duration, such as a track containing only one key.

When per-track looping is disabled, requests outside the keyed range should simply hold the nearest endpoint pose. The animation panel also provides timeline-wide looping, which controls playback across the editor's timeline. That control is already implemented; your responsibility here is the individual track's loop flag.

5.5 Easing Curves

Linear interpolation moves through an interval at a constant rate. Easing allows the same motion to begin slowly, arrive gradually, or hold a pose before jumping to the next one. You will implement this behavior in **applyEasing(int easing, float u)** by remapping the interpolation parameter before it is used to calculate the pose.

Clamp the input to **[0, 1]**, then apply the curve corresponding to the selected mode:

- **EASE_LINEAR** returns u, producing constant speed.
- **EASE_IN** returns u^2, so the motion starts slowly and accelerates.
- **EASE_OUT** returns $1 - (1 - u)^2$, so the motion slows as it approaches the next key.
- **EASE_IN_OUT** returns $3u^2 - 2u^3$, producing a gradual start and finish.
- **EASE_STEP** returns 0 while $u < 1$ and returns 1 when $u = 1$, holding the starting pose until the next key is reached.

Every mode must preserve the endpoints: $f(0) = 0$ and $f(1) = 1$. If the function receives an unrecognized easing mode, use linear behavior.

The easing setting belongs to the key at the beginning of an interval. In other words, key **i** controls how the animation progresses toward key **i + 1**. The final key has no outgoing interval, so its easing setting does not affect the motion.

As you test these curves, notice that easing changes when the shape reaches intermediate poses while preserving the keyed endpoints. You should be able to change an interval's easing and see a difference in its timing without changing where the motion begins or ends.

5.6 Understanding the Rotation Limitation

Taking the shorter angular difference on each axis resolves the full-turn boundary problem, but it does not solve every difficulty involved in interpolating rotations. When several Euler angles change together, interpolating them independently does not generally produce the shortest rotation path in three dimensions. The resulting motion may also change angular speed even when the interpolation parameter advances at a constant rate.

Euler angles also have singular configurations in which two rotation axes align and a degree of freedom is lost. This limitation is commonly called *gimbal lock*. You are not required to replace Euler angles with quaternions or implement spherical interpolation for this assignment, but you should understand why the method you are implementing has limitations.

Assignment 4

You encountered a related orientation problem in the camera assignment. When the camera looks directly along the world up direction, the look-at calculation cannot construct a valid camera basis. Clamping the pitch away from the poles prevents that configuration. The camera's degenerate basis and Euler-angle gimbal lock arise in different calculations, but both illustrate why certain orientations require care when representing and manipulating rotations.

6 Testing Your Assignment

Test your implementation directly in the editor as you complete each part of the assignment. Begin with the operations that add, remove, and move keys, then check interpolation, easing, and looping. This order will help you identify the source of a problem, since incorrect key storage can affect the animation calculations that depend on it.

Choose simple poses with clearly visible differences so that you can predict what should happen between them. Once those examples behave correctly, combine the features in a more complete scene and verify that the animation survives saving and loading.

6.1 Testing Keyframe Storage

Begin by inserting a shape and creating several poses at different times using **Set key at t**. Add the keys out of chronological order—for example, create keys at 4 seconds, 0 seconds, and then 2 seconds. Confirm that the timeline places them in the correct order and that selecting or scrubbing to each key displays the expected pose.

Next, return to an existing key time, change the shape's pose, and set another key. The new pose should replace the previous one without adding another diamond at the same time.

To check deletion, place two keys close together and position the playhead slightly closer to one of them. Use **Delete key** and confirm that the nearer key is removed. Also try deleting when no key is close to the playhead; the track should remain unchanged.

Test retiming by dragging a key past its neighbors in both directions. Continue dragging after the keys change order and verify that the same pose remains attached to the selected diamond. Then drag a key onto another key. The moved key should replace the destination key while retaining its own pose and easing setting. Finally, try dragging a key toward a negative time and confirm that it stops at zero.

6.2 Testing Interpolation and Easing

Create a simple track with two noticeably different poses. For example, place a shape at (0, 0, 0) at 0 seconds and at (4, 2, 0) at 2 seconds. With linear easing selected, move the playhead to 1 second. The shape should be halfway between the keyed positions, at (2, 1, 0).

Repeat this process with scale and rotation. To check rotation across the zero-degree boundary, create a rotation of 350 degrees on one axis at the first key and 10 degrees at the second. The shape should rotate through the shorter 20-degree arc. Try negative angles and angles greater than 360 degrees as well, remembering that different angle values can represent the same visible orientation.

Compare all five easing modes using the same pair of keys. Linear should produce constant motion, Ease In should begin slowly, Ease Out should slow toward the destination, and Ease In-Out should do both. Step should hold the starting pose until the playhead reaches the next key, then switch to that key's pose. Change the easing on the key that begins the interval, since that setting controls the motion toward the following key.

For every mode, check the poses at both key times. Changing the easing should affect the timing between the keys while preserving the poses you recorded.

6.3 Testing Boundaries and Looping

Before enabling looping, check how the editor behaves outside a track's keyed range. Create keys at 2 and 5 seconds, then move the playhead before 2 seconds and after 5 seconds. The shape should hold the first pose before the track begins and the last pose after it ends.

Now enable per-track looping and repeat the test. Because this track lasts 3 seconds, a request at 6 seconds should produce the same pose as a request at 3 seconds. A request at 1 second should match the pose at 4 seconds. At exactly 5 seconds, the shape should still display the final keyed pose.

Check a shape with only one key as well. It should retain that pose wherever you move the playhead, including when looping is enabled. A shape with no animation track should remain unaffected by timeline playback.

Keep per-track looping separate from the panel's timeline-wide loop control while testing. If playback keeps returning the playhead to the beginning of the timeline, use direct time entry to inspect the individual track at the times you want to compare.

6.4 Testing Playback and Scene Persistence

Once the individual features behave correctly, play the animation and watch how they work together. Scrub forward and backward, jump between keys, and enter times directly. The pose at a particular time should be consistent regardless of how you reached it.

Save your scene, reopen it, and play the animation again. Confirm that the shapes retain their tracks, key times, poses, easing settings, and looping behavior. If your scene uses imported assets, verify that those assets still load correctly.

Finish by watching the scene you intend to submit. Check that its motion is easy to follow and that it visibly demonstrates the required translation, rotation, scale, easing, and looping behavior. Record any remaining problems in your README, including how to reproduce them and what you think is causing them.

7 Create a Scene, Save It and Record a Video

Once your animation features are working, use them to create an original animated scene. This is your opportunity to bring the individual parts of the assignment together and demonstrate how changes in position, rotation, scale, and timing can produce a coordinated sequence. Choose a clear idea for the motion and arrange the scene so that the important actions are easy to see.

7.1 Create Your Animated Scene

Your scene must include:

- At least three animated shapes, each with its own track.
- At least four keys on one of those tracks.
- Animation across all three transform channels: something translates, something rotates, and something changes scale.
- At least two different easing curves used where their timing differences are visible.
- At least one track with per-track looping enabled.

These requirements may be distributed across the shapes; every shape does not need to demonstrate every feature. For example, one shape might move between positions while another rotates and a third changes scale. Think about how their actions relate to one another so that the result feels like a scene rather than a collection of unrelated tests.

Choose a camera position that keeps the important motion visible throughout playback. Pay attention to the timing of each action and give the viewer enough time to recognize the easing differences. If a loop is intended to repeat smoothly, arrange its first and last poses accordingly. A visible jump can also be intentional, but it should serve the motion you are trying to create.

Make an awe-inspiring scene and your video could make it to the CP 484 Hall of Fame.

7.2 Save and Verify Your Scene

Save your animation through **File > Save Scene** as `animation.scene`. The scene file stores the tracks, keys, easing settings, and looping information needed to replay your work.

Reopen the saved scene in your final build and watch the animation again. Confirm that the saved version behaves as expected and still meets every requirement in Section 7.1.

If your scene uses imported meshes or other external assets, include those files with your submission. Test the project from a fresh folder to confirm that the asset paths remain valid and that the scene can be opened without relying on files elsewhere on your computer.

7.3 Record a Video

Record a screen video showing your completed animation running in the editor. Begin with the scene loaded and the editor window visible, then play the animation long enough to show the required motion and at least one repetition of the looping track.

Make sure the recording clearly shows translation, rotation, changes in scale, and the timing differences produced by your easing choices. You may replay an interval or briefly demonstrate timeline scrubbing if it helps explain a feature. Keep the recording focused on the completed scene; a walkthrough of your source code is not necessary.

Check the video before submitting it to make sure it plays correctly and the animation is easy to see. Submit it in a commonly playable format, such as MP4. The video demonstrates the behavior of your implementation, while `animation.scene` allows me to load and inspect the animation directly. Include both.

8 Submission

Submit one compressed archive containing your implementation, saved scene, video, and supporting files. Before creating the archive, build the editor from a clean state and complete the manual checks described in Section 7. Your final executable must use your own animation implementation.

8.1 Source Code and Executable

Complete the seven required functions in `src/Keyframe.cpp` and make sure your Identity is updated under **File > Options**.

Include your source code and the compiled editor executable for your development environment. You may submit the complete project or only the files you changed. If you submit changed files only, explain in your README how to integrate them into the supplied starter project, and verify those instructions using a clean copy.

You may use supplied solution libraries for other assignment topics when available. Identify the libraries and build options in your README. Do not use ANIMATION=solution for your final build, since that option replaces the animation code you are required to implement.

8.2 README

As a final step, create a README.txt or README.md file containing the following information:

- Name of student
- Operating system/CPU architecture/development environment used. Include version of OS and architecture type, as this helps me with compilation issues.
- Special instructions on how to compile and run your code if different from the instructions provided.
- Did you collaborate with anyone in the class? If so, describe who you worked with and what assistance was given or received.

- Were there any references, books, websites, or other resources that you found particularly useful? Please list them.
- An explanation of why moveKey() returns a new index and what can happen if it returns the original index.
- An explanation of gimbal lock in your own words and how the camera's pole safeguard relates to orientation singularities.
- Are there any known problems with your implementation? If so, describe them.
- A short guide identifying which shapes demonstrate translation, rotation, scale, easing, and looping. Identify the track with at least four keys and the intervals where the easing differences are clearest.
- Any known problems, how to reproduce them, and what you think is causing them.
- Any additional functionality or comments about the assignment.

8.3 Submission Checklist

Your archive must contain:

- Your source code and the project files needed to build it.
- Your compiled editor executable.
- *animation.scene*.
- Any external assets needed to load the scene.
- Your recorded demonstration video.
- Your README.txt or README.md.

Use either ZIP or TAR.GZ format and submit one archive. Before submitting, extract it into a fresh folder, follow your own build instructions, open the saved scene, and check that the video plays correctly. This final check should confirm that the submission contains everything needed to compile, run, and review your work.